Приднестровский государственный университет им. Т.Г. Шевченко
Физико-технический институт

ПРАКТИЧЕСКИЙ КУРС ВЕБ РАЗРАБОТКИ REACT TYPESCRIPT NODE И SUPABASE

Лабораторный практикум

Разработал:
ст. преподаватель
кафедры ИТ
Бричаг Д.В.

г. Тирасполь
2026 г.

Лабораторная работа 4

Загрузка справочника приложения по HTTP

Продолжите проект третьей лабораторной. Вместо свободного ввода тега пользователь будет выбирать значение справочника, загруженного по сети. Форма покажет ожидание, ошибку или отсутствие вариантов и позволит повторить запрос без потери черновика. Создание, редактирование, удаление и фильтр предыдущей работы сохраняются.

Цель работы

Научиться получать JSON через fetch, проверять ответ во время выполнения и связывать сетевую загрузку с интерфейсом React. Отделить HTTP-сервис от формы, управлять состояниями запроса и не применять результат уже отменённой загрузки.

Обязательный результат

Справочник своей темы находится в frontend/public/demo и запрашивается по HTTP. Запрос виден в Network, полученные варианты используются в форме создания и редактирования. Есть проверка HTTP-статуса и структуры JSON, состояния загрузки, ошибки и пустого результата, кнопка повторной попытки. Пока обязательный справочник недоступен или значение не выбрано, сохранить запись нельзя.

Источник на этом этапе — JSON собственного проекта, который раздаёт Vite. Отдельный сервер, коммерческий API и ключи не нужны. Основные записи остаются в состоянии App и после перезагрузки возвращаются к демонстрационному набору. Пояснение об этом из ЛР 3 остаётся в интерфейсе. В ЛР 5 источник справочника будет заменён на Supabase.

Теоретическая справка

Запрос и проверка ответа

HTTP связывает клиента и сервер: браузер отправляет GET по адресу ресурса и получает статус, заголовки и тело ответа. JSON описывает данные, но сам по себе не задаёт их прикладную структуру. Файл из public доступен по URL; импорт файла в TypeScript не является сетевой загрузкой.

fetch возвращает Promise. await позволяет дождаться ответа внутри async-функции, а response.json() отдельно читает и разбирает тело. Ответ с кодом 404 или 500 нужно обработать через response.ok: получение такого ответа само по себе не отклоняет Promise fetch. Сбой соединения и неверный JSON также требуют обработки.

Тип Tag существует для проверки программы TypeScript. Запись as Tag[] не проверяет данные сервера. Примите результат как unknown, убедитесь, что это массив объектов с нужными полями, и только затем передавайте его форме. Пустой корректный массив и ответ с неправильной структурой — разные ситуации.

Жизненный цикл загрузки

useEffect запускает работу с внешним источником после появления компонента. При завершении его жизни или смене зависимостей функция очистки прекращает прежнюю загрузку. В этой работе AbortController отменяет запрос, а дополнительный флаг не позволяет устаревшему обработчику изменить состояние.

У сетевого справочника своё состояние, у формы — свой черновик, у App — основной список. Повторный запрос не должен стирать введённое название или создавать записи. Хук useTags объединит только загрузку и её состояния; TaskForm продолжит отвечать за ввод и сохранение.

Практическая часть

Шаг 1 Подготовьте проект и выберите справочник

Сохраните завершённую ЛР 3 в Git. Из frontend выполните npm ci и npm run build, затем запустите npm run dev. Проверьте создание, редактирование, отмену и удаление записи. Новые зависимости в этой работе не требуются.

Примеры ниже продолжают Task Tracker: справочник содержит теги, у задачи выбирается один тег. Для других тем заменяйте Tag, tags и поле tag своим справочником. Выберите значения, которые уже нужны форме, и сохраните смысл остальных полей.

1. Task Tracker — теги задач.
2. Mini Issue Tracker — типы issue, например bug и task.
3. Expense Tracker — категории операций.
4. Фильмы и сериалы — жанры.
5. Игры — платформы.
6. Планировщик мероприятий — категории событий.
7. Habit Tracker — категории привычек.
8. Учебный менеджер — курсы.
9. Библиотека — жанры.
10. Personal CRM — вымышленные компании.

Для справочника типов issue достаточно двух значений; для остальных подготовьте минимум пять. Используйте постоянный строковый id и понятное пользователю name. Изменение названия не должно менять id. Эти идентификаторы понадобятся при переносе справочника в БД.

Шаг 2 Создайте JSON и проверьте его адрес

Создайте frontend/public/demo/tags.json. В JSON обязательны двойные кавычки, нельзя оставлять комментарии и запятую после последнего элемента. Для Task Tracker используйте, например, такой набор:

```
[
  { "id": "study", "name": "Учёба" },
  { "id": "work", "name": "Работа" },
  { "id": "home", "name": "Дом" },
  { "id": "health", "name": "Здоровье" },
  { "id": "personal", "name": "Личное" }
]
```

При стандартном base Vite откройте /demo/tags.json на том же адресе и порте, где работает приложение. Например, если Vite сообщил http://localhost:5173, полный адрес будет http://localhost:5173/demo/tags.json. Папка public в URL не пишется. Проверяйте адрес своего процесса, поскольку порт может отличаться.

Убедитесь, что видите массив, а не HTML приложения. Данные из public доступны посетителям: здесь находятся только учебные справочные значения. Редактирование формы не изменяет этот файл; сервер Vite раздаёт его для чтения.

Шаг 3 Опишите контракт и согласуйте прежние записи

Создайте frontend/src/types/tag.ts. Тип описывает один вариант выбора:

```
export type Tag = {
  id: string
  name: string
}
```

В Task оставьте поле tag типа string, но теперь храните в нём id справочника, например study, а не отображаемое имя Учёба. TaskDraft из ЛР 3 автоматически наследует это поле. В пустом черновике оставьте tag: пустая строка, чтобы пользователь явно выбрал значение.

В демонстрационных задачах frontend/src/data/tasks.ts замените прежние произвольные теги на существующие id. Другие поля и id задач сохраните. Перезапустите сеанс страницы один раз, чтобы App получил обновлённый набор; это ожидаемо сбросит временные изменения ЛР 3. Уже введённый в текущей форме черновик при дальнейших повторах запроса сбрасывать нельзя.

На этом этапе список и карточка могут показывать сохранённый код с подписью «Тег». В форме отображается name, а value содержит id. Если выводите название на других страницах, получайте его из того же справочника; не создавайте вторую вручную заполненную таблицу названий.

Шаг 4 Проверьте структуру внешних данных

Создайте frontend/src/services/tags.ts. Начните с импорта типа и проверки одного элемента. Условие проверяет объект до обращения к его полям:

```
import type { Tag } from '../types/tag'

function isTag(value: unknown): value is Tag {
  if (typeof value !== 'object' || value === null) return false
  return (
    'id' in value && typeof value.id === 'string' &&
    value.id.trim().length > 0 && value.id === value.id.trim() &&
    'name' in value && typeof value.name === 'string' &&
    value.name.trim().length > 0
  )
}
```

В том же файле добавьте разбор массива. Для select и React key идентификаторы должны быть уникальны. Пустой массив здесь допустим: его отдельно обработает интерфейс. Лишние поля ответа можно игнорировать.

```
export function parseTags(value: unknown): Tag[] {
  if (!Array.isArray(value) || !value.every(isTag)) {
    throw new Error('Неверная структура справочника тегов.')
  }
  const tags: Tag[] = value
  const ids = new Set(tags.map((tag) => tag.id))
  if (ids.size !== tags.length) {
    throw new Error('В справочнике повторяются идентификаторы.')
  }
  return tags
}
```

Проверка value is Tag связывает проверку во время выполнения с сужением типа TypeScript. Не заменяйте её утверждением as Tag[] и не отключайте strict. Если ваша тема требует дополнительные поля справочника, добавьте их в тип и проверку.

Шаг 5 Вынесите HTTP в сервис

В конце services/tags.ts добавьте getTags. BASE_URL учитывает настроенный базовый путь Vite. При стандартных настройках получится /demo/tags.json. signal будет передан из компонента для отмены запроса.

```
const tagsUrl = `${import.meta.env.BASE_URL}demo/tags.json`

export async function getTags(signal: AbortSignal): Promise<Tag[]> {
  const response = await fetch(tagsUrl, {
    signal,
    cache: 'no-store',
  })
  if (!response.ok) {
    throw new Error(`Не удалось загрузить теги: HTTP ${response.status}`)
  }
  const value: unknown = await response.json()
  return parseTags(value)
}
```

cache: no-store упрощает учебную проверку после изменения JSON. Сервис возвращает данные или отклонённый Promise; он не показывает alert, не меняет App и не подставляет скрытый резервный массив. Ошибка не должна выглядеть как успешная загрузка пустого справочника.

Не вызывайте getTags непосредственно в теле компонента во время рендера и не импортируйте tags.json как модуль. В Network должен быть отдельный запрос из приложения. Даже если запрос вернул HTTP 200, HTML вместо JSON или неправильные поля должны привести к ошибке.

Шаг 6 Свяжите загрузку с жизненным циклом формы

Создайте frontend/src/hooks/useTags.ts. Этот пользовательский хук вызывается один раз на верхнем уровне TaskForm. Сначала объявите состояние запроса.

Три варианта типа позволяют читать data только после успешного получения ответа.

```
import { useEffect, useState } from 'react'
import type { Tag } from '../types/tag'
import { getTags } from '../services/tags'

type TagsLoad =
  | { status: 'loading' }
  | { status: 'error'; message: string }
  | { status: 'success'; data: Tag[] }

export function useTags() {
  const [load, setLoad] = useState<TagsLoad>({ status: 'loading' })
  const [attempt, setAttempt] = useState(0)
```

Следующий фрагмент продолжает ту же функцию useTags. Добавьте его сразу после состояний; он завершает хук. useEffect получает обычную функцию, а асинхронная работа выполняется через Promise сервиса.

```
  useEffect(() => {
    const controller = new AbortController()
    let active = true
    setLoad({ status: 'loading' })
    getTags(controller.signal)
      .then((data) => {
        if (active) setLoad({ status: 'success', data })
      })
      .catch((error: unknown) => {
        if (!active) return
        const message = error instanceof Error
          ? error.message : 'Не удалось загрузить справочник.'
        setLoad({ status: 'error', message })
      })
    return () => {
      active = false
      controller.abort()
    }
  }, [attempt])

  function retry() {
    setLoad({ status: 'loading' })
    setAttempt((current) => current + 1)
  }
  return { load, retry }
}
```

Изменение attempt запускает новую попытку. Очистка закрывает предыдущую: флаг active относится к конкретному запуску эффекта, а не ко всему приложению. При уходе со страницы TaskForm размонтируется, и отменённый запрос не покажет ошибку на другой странице.

В режиме разработки StrictMode может выполнить дополнительный цикл запуска и очистки эффекта. В Network допустим отменённый первый запрос и успешный следующий. Не удаляйте StrictMode и не скрывайте проблему глобальным флагом «уже загружено». Проверьте, что в итоге форма получает актуальные данные.

Шаг 7 Замените поле тега сетевым выбором

В TaskForm.tsx импортируйте useTags из ../hooks/useTags. Внутри TaskForm, рядом с состояниями draft и error из ЛР 3, вызовите хук и вычислите доступность выбора и сохранения:

```
const { load, retry } = useTags()
const tags = load.status === 'success' ? load.data : []
const selectedTagExists = tags.some((tag) => tag.id === draft.tag)
const canSave = load.status === 'success' && selectedTagExists
```

Удалите прежний текстовый input для tag. Внутрь существующей формы добавьте подпись и select. Остальные поля, обработчики onSave и onCancel, а также key={task.id} у формы редактирования сохраните.

```
<label htmlFor="task-tag">Тег</label>
<select id="task-tag" value={draft.tag} required
  disabled={load.status !== 'success' || tags.length === 0}
  onChange={(event) => setDraft({
    ...draft, tag: event.target.value,
  })}>
  <option value="">Выберите тег</option>
  {draft.tag !== '' && !selectedTagExists && (
    <option value={draft.tag} disabled>
      Текущий код: {draft.tag} — выберите доступный тег
    </option>
  )}
  {tags.map((tag) => (
    <option key={tag.id} value={tag.id}>{tag.name}</option>
  ))}
</select>
```

Дополнительный option сохраняет видимым прежний код, если он пока не найден. Не заменяйте его автоматически первым вариантом: это незаметно изменило бы редактируемую запись. После успешной загрузки пользователь выбирает существующий тег сам.

Шаг 8 Покажите загрузку и повторную попытку

Сразу после select, внутри той же формы, выведите состояния. Кнопки повторной попытки имеют type="button", поэтому не вызывают сохранение. Пустой результат получает своё объяснение и возможность повторить загрузку после исправления файла.

```
{load.status === 'loading' && (
  <p role="status">Загружаем теги…</p>
)}
{load.status === 'error' && (
  <div>
    <p role="alert">{load.message}</p>
    <button type="button" onClick={retry}>Повторить</button>
  </div>
)}
{load.status === 'success' && tags.length === 0 && (
  <div>
    <p role="status">Тегов пока нет. Сохранение недоступно.</p>
    <button type="button" onClick={retry}>Повторить</button>
  </div>
)}
```

Пока запрос выполняется, повторная попытка не показывается. Сохраните текстовые поля и кнопку отмены на экране во всех состояниях. Не заменяйте всю форму экраном загрузки и не меняйте её key при каждом запросе: повтор должен сохранять черновик.

При успешной непустой загрузке с неизвестным прежним тегом покажите отдельное сообщение о необходимости выбрать доступный вариант. Название, дата и другие поля при этом остаются прежними. Для проверки этого случая можно временно убрать выбранный тег из JSON и открыть редактирование заново без перезагрузки приложения.

Шаг 9 Защитите сохранение и проверьте продолжение проекта

В handleSubmit сразу после event.preventDefault() добавьте проверку canSave. Проверки названия, даты и предметного правила из ЛР 3 оставьте после неё. Существующий вызов props.onSave выполняется только после всех проверок.

```
if (!canSave) {
  setError('Дождитесь справочника и выберите доступный тег.')
  return
}
```

У существующей кнопки «Сохранить» добавьте disabled={!canSave}. Сообщение error и кнопка отмены из ЛР 3 сохраняются. Проверка в обработчике нужна вместе с disabled, поскольку решение о допустимости записи должно проверяться в самой операции сохранения.

```
<button type="submit" disabled={!canSave}>Сохранить</button>
<button type="button" onClick={props.onCancel}>Отмена</button>
```

Форма должна работать на обоих маршрутах — создания и редактирования. В карточке сохраняется выбранный id тега; повторное редактирование показывает его название, если справочник содержит этот id. Существующий фильтр по статусу остаётся. Добавлять второй фильтр обязательно только позже; если добавляете фильтр по тегу сейчас, его варианты тоже получайте через сервис.

Недоступность тегов блокирует сохранение формы, но не просмотр локального списка, переходы, удаление с подтверждением или отмену. Не переносите основной список из App в хук загрузки и не обнуляйте tasks после ошибки справочника.

Шаг 10 Проверьте запросы и нештатные ответы

Откройте инструменты разработчика браузера и вкладку Network. Для повторяемых проверок включите Disable cache, держите инструменты открытыми и при необходимости выберите ограничение скорости. Фильтруйте запросы по

tags.json. HTTP-статус, Response и Initiator должны подтверждать, что данные запросил ваш сервис.

1. Откройте создание задачи. Увидьте GET, проверьте массив в Response, выберите тег и сохраните запись. Откройте редактирование и убедитесь, что выбор соответствует сохранённому id.
2. Включите медленную сеть, уйдите к списку и снова откройте форму. Во время загрузки видны сообщение и недоступная кнопка сохранения, остальные поля доступны.
3. В Network временно включите Offline перед открытием формы. Дождитесь ошибки, заполните название, верните Online и нажмите «Повторить». Черновик должен сохраниться.
4. Временно измените путь tagsUrl на отсутствующий файл и откройте форму. Должна появиться ошибка. Если сервер вернул HTML с кодом 200, ошибка разбора JSON также корректна. Верните путь.
5. Замените содержимое tags.json на [], откройте форму и увидьте отдельное пустое состояние. Восстановите массив и нажмите «Повторить».
6. Верните JSON-объект вместо массива, затем массив с числовым id или отсутствующим name. Каждый ответ должен завершиться ошибкой структуры. Отдельно проверьте повторяющийся id.
7. На медленной сети откройте форму и сразу уйдите к списку. Устаревший ответ не должен менять другую страницу. Снова откройте форму и дождитесь её собственного запроса.
8. Восстановите исходный JSON и обычную сеть. Проверьте весь CRUD, отмену и фильтр ЛР 3. Изменение тега в черновике с последующей отменой не меняет исходную запись.

Для шага с Offline сначала загрузите приложение, затем меняйте режим сети: иначе можно заблокировать сам frontend. После каждого эксперимента воз-

вращайте нормальные данные и настройки. Если изменение файла вызвало полную перезагрузку через Vite, повторите проверку сохранности черновика через Offline → Online, не редактируя исходники.

Обновление страницы по-прежнему сбрасывает локальные задачи. Это отличается от кнопки «Повторить», которая загружает только справочник и не перезапускает приложение. Не используйте window.location.reload() для повторной попытки.

Шаг 11 Сохраните воспроизводимый результат

Остановите dev и из frontend выполните npm run build, затем npm run preview. В готовой сборке проверьте URL справочника, создание и редактирование с выбором тега. Vite должен включить public/demo/tags.json в результат сборки. Не исправляйте вручную файл в dist: исходником остаётся public/demo/tags.json.

В README опишите источник справочника и временное хранение основных записей. В docs/progress.md добавьте ЛР 4 и точные действия для проверки загрузки, ошибки, пустого ответа, неверной структуры и повтора без потери черновика. Укажите фактические пути для своей темы.

```
git status
git add .
git diff --cached --stat
git commit -m "Lab 4: load reference data over HTTP"
git push
```

Команды Git выполняйте из корня репозитория. Перед коммитом восстановите корректный JSON и рабочий URL; уберите временные проверки из интерфейса. В чистой копии проекта выполните npm ci и npm run build из frontend. Преподавателю передайте ссылку на тот же репозиторий и коммит ЛР 4.

Частые проблемы

Вместо JSON приходит HTML. Проверьте путь: файл лежит в public/demo, а URL содержит demo без public. Откройте Response в Network. Наличие HTTP

200 не доказывает правильность тела; не скрывайте ошибку разбора резервными данными.

Запрос повторяется бесконечно. Проверьте зависимости useEffect: load и новый объект параметров не должны вызывать новую загрузку после каждого обновления состояния. В примере повтор зависит только от attempt. Не вызывайте retry во время рендера.

После повторной попытки исчезло название. Проверьте, что TaskForm остаётся смонтированной, её key не зависит от состояния сети, а setDraft не вызывается в эффекте загрузки. Справочник не является начальным значением всей формы.

В select видны названия, но редактирование не выбирает нужное. Сравните task.tag и option value: оба должны содержать одинаковый строковый id. Проверьте демонстрационные данные, пробелы и случай, когда значение удалено из справочника.

Кнопка активна при пустом справочнике. Проверяйте не только статус success, но и наличие выбранного id в загруженном массиве. При [] вариантов нет. Ошибка сети не должна превращаться в success с пустым массивом.

В консоли появляется ошибка отмены. Очистка эффекта сначала делает active равным false, затем отменяет запрос. catch игнорирует результат завершённого запуска. Это ожидаемая отмена при уходе, а не повод показывать пользователю сбой.

Использование ИИ при выполнении работы

Можно попросить ИИ объяснить конкретный ответ Network, проверку unknown или жизненный цикл вашего эффекта. Передавайте небольшой фрагмент сервиса и состояния загрузки. Самостоятельно воспроизведите ошибку, пустой ответ и повтор; объясните, почему асинхронные данные нельзя считать корректными только на основании типа TypeScript.

Результаты работы и сдача

В репозитории находятся JSON-справочник, его тип, HTTP-сервис с проверкой ответа, хук загрузки и обновлённая форма. Состояния ожидания, ошибки и пустого результата различимы. Повторная попытка сохраняет введённые поля. Прежние страницы и локальные операции продолжают работать.

На демонстрации покажите запрос в Network, выберите значение в форме и создайте запись. Затем воспроизведите сбой, восстановите сеть и повторите запрос без потери черновика. Покажите [] и неправильную структуру ответа. В конце восстановите корректный справочник и продемонстрируйте редактирование, отмену и удаление.

Критерии оценки

Максимум — 10 баллов. Синхронный импорт JSON или отдельный экран со справочником, который не используется в форме проекта, не выполняет обязательный результат.

Функционал — 5 баллов: реальный GET к справочнику из public (1); отдельный сервис с проверкой response.ok (1); проверка структуры и уникальности id (1); выбор из загруженных данных при создании и редактировании (1); повторная попытка без перезагрузки и потери черновика (1).

Ошибки и крайние случаи — 2 балла: различимые loading, error и empty, запрет сохранения без допустимого выбора (1); отмена или игнорирование устаревшего результата, корректная обработка неизвестного тега (1).

Непрерывность проекта — 1 балл: CRUD, отмена, фильтр, маршруты и временный характер основных записей из ЛР 3 сохранены; справочник включён в прежний пользовательский сценарий.

Git и воспроизводимость — 2 балла: корректные исходники и коммит доступны преподавателю (1); README и журнал этапа содержат проверки, чистая копия собирается, справочник доступен в preview (1).

Вопросы для защиты

1. Чем запрос к JSON из public отличается от его импорта в TypeScript?
2. Почему после fetch нужно проверить response.ok?
3. Чем неправильная структура ответа отличается от пустого массива?
4. Почему as Tag[] не проверяет данные сервера?
5. Где живут основной список, черновик и состояние загрузки?
6. Что делает очистка useEffect при уходе со страницы?
7. Как повторная попытка сохраняет введённые поля?
8. Почему в записи хранится id справочника, а в select показывается name?

Материалы для самостоятельной работы

React — useEffect и очистка эффекта: https://react.dev/reference/react/useEffect

MDN — использование Fetch API и отмена запроса: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Vite — статические ресурсы и каталог public: https://vite.dev/guide/assets.html

Документация проверена при подготовке работы. Практические шаги и критерии относятся к последовательности нашего курса.